Where should extra compute go in a diffusion language model? Timestep versus state in CoBit-S

Short research report · CoBit-S (130M) on LM1B · counterfactual adaptive-compute study · October 2026
Summarises *experiments/adaptive_compute/report/REPORT.md* (commit f136ced); all numbers are from that study's result files.

1. Executive summary

We asked whether CoBit-S would benefit from spending inference compute *adaptively*: giving extra denoiser evaluations to the states that need them most, rather than spending compute uniformly. Using counterfactual branches from saved denoising states, we measured how much one more unit of compute changes the final sample's quality, at different points in the trajectory and for different samples.

- **Compute value depends strongly on the timestep.** The same extra compute lowers GPT-2 NLL by up to 0.39 nats/token when spent in the middle of denoising (steps 64–127), but does almost nothing early and slightly hurts late (after step 160).
- **Compute value depends only weakly on the individual state.** Within a timestep, samples differ measurably at only one of eight checkpoints, the differences are hard to predict from cheap state features, and no state-aware allocation beats a timestep-only schedule.
- **The NLL gains are confounded with entropy.** Every large gain comes with lower sample entropy (up to −0.17 nats), so we cannot yet call the mid-trajectory gains quality improvements.

	What the experiments show	Verdict
(a) **Timestep-dependent** allocation (fixed schedule over denoising steps)	Large, consistent differences between timesteps; a timestep-only schedule captures all the allocation gain we could find.	**Supported** (gains entropy-confounded)
(b) **State-dependent or token-level** allocation (routing by sample, state or token)	Small per-state signal at one checkpoint; within-timestep rank correlation 0.03–0.06; no gain over (a). Token-level allocation not tested.	**Not justified** by current data

2. Experimental setup

Model and sampler. CoBit-S on LM1B with the production stochastic sampler (255 model evaluations, NFE, per sample). We took 1,000 unconditional samples and saved each one's state at 8 checkpoints (the start of steps 0, 32, …, 224; noise level σ from 77 down to 0.34).

Unit of extra compute. "+c" adds c extra self-conditioning passes at every step of the 32-step window after a checkpoint (+32c NFE, i.e. +12.5% of baseline compute per unit). The noise schedule, the update rule and the random noise are unchanged, so the branch differs only in compute. We tested +1, +2 and +4.

Quality and noise control. Quality Q is the negative GPT-2-large NLL per token of the final sample (higher is better). Because a single continuation is dominated by trajectory randomness, each state and compute level was continued 4 times with shared random numbers across compute levels. Two independent halves of these continuations let us separate real differences between states from noise; predictors and allocation policies were chosen on one half and scored on the other. Total cost: 3.3 GPU-hours on one RTX 4090.

3. Main finding: compute value varies substantially by timestep

The average benefit of extra compute changes sharply along the trajectory (Figure 1). With +4 (+128 NFE), mean Q improves by **0.393** nats/token [95% CI 0.368, 0.418] at checkpoint 2 and **0.278** [0.260, 0.297] at checkpoint 3, by 0.068 at checkpoint 4 and by only 0.002–0.048 at checkpoints 0–1. From checkpoint 5 on, extra compute slightly *lowers* Q (−0.007 to −0.037). Returns diminish: at checkpoint 3 the first unit adds 0.162, the second 0.068.

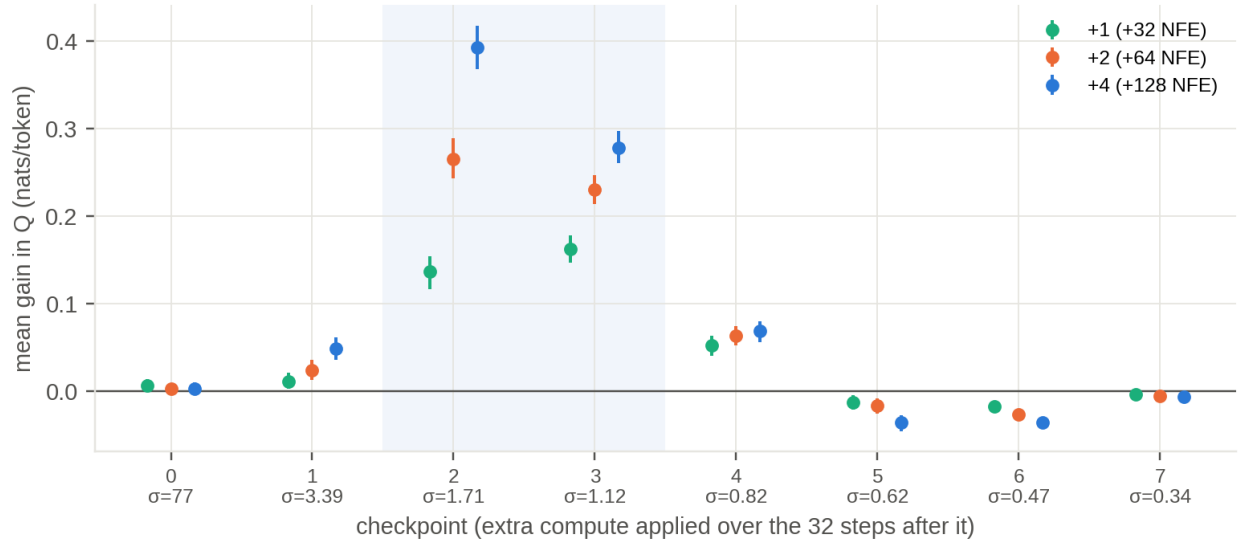


Figure 1. Mean gain in Q from extra self-conditioning compute applied in the 32 steps after each checkpoint (1,000 states, 95% CIs). The benefit is concentrated in the middle of denoising (shaded, steps 64–127) and is near zero or negative elsewhere. Gains here are partly entropy reductions (Section 5).

End-to-end runs over all 1,024 samples confirm the pattern at matched compute. Spending +4 only in checkpoints 2–3 (511 NFE) lowered GenPPL from 60.2 to **36.4** (ΔQ 0.510 [0.464, 0.556]); spreading the same budget as +1 everywhere (509 NFE) reached 45.8 (ΔQ 0.276 [0.236, 0.314]). Gains from different checkpoints do not fully add up: the combined runs delivered 76% and 83% of the sum of the per-checkpoint effects.

4. State-dependent compute: evidence is weak and not practically exploitable

For state-dependent allocation to help, three things must hold: states at the same timestep must differ in how much they gain, those differences must be predictable, and acting on the prediction must beat a simpler schedule. The evidence is weak on the first point and negative on the other two.

- **Opportunity is small.** The spread of gains between states (τ, after removing noise) is detectable only at checkpoint 3: τ = 0.120 [0.071, 0.153] for +4, against a mean gain of 0.278. At the other seven checkpoints the confidence intervals include zero (Figure 2). Two apparent effects in the 200-sample pilot did not replicate at 1,000 samples. Even at checkpoint 3, measurement noise (0.26) is twice τ.
- **Predictability is weak.** Ridge predictors on 15 cheap state and trajectory features rank states within a timestep with Spearman ρ = 0.03–0.06 (CIs exclude zero, but the signal is tiny). The timestep alone gives a pooled ρ of 0.37 for +4, and state features add nothing to it. Trained without the timestep and tested on unseen checkpoints, state features transfer poorly (ρ = 0.12, negative R^2).
- **No allocation gain over timestep.** At equal compute, a learned state-plus-timestep allocator matches the best timestep-only schedule (difference +0.003 [−0.005, +0.011] nats/token at +100% compute; −0.008 to +0.004 across all budgets). Giving whole samples more compute is no better than random (+0.000 [−0.029, +0.031]), and picking individual states from noisy measurements is clearly worse (Figure 3). Token- and bit-level allocation could not be tested: the sampler has no mechanism to spend compute on part of a sequence.

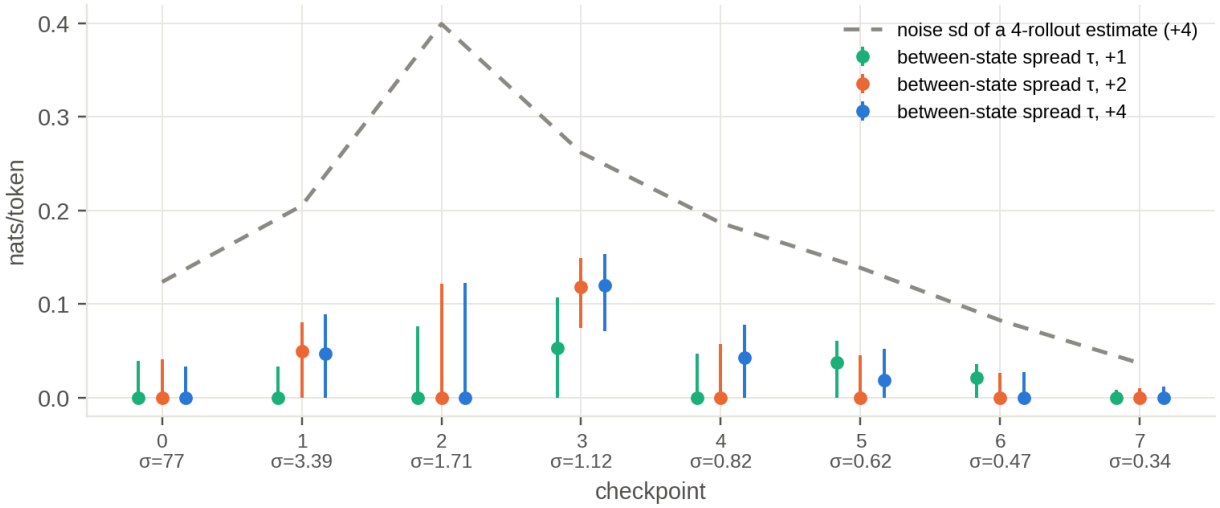


Figure 2. Between-state spread of the gain (τ, noise removed by split-half covariance; 95% CIs) versus the noise in a 4-continuation estimate. Only checkpoint 3 shows a spread clearly above zero, and it is well below the measurement noise.

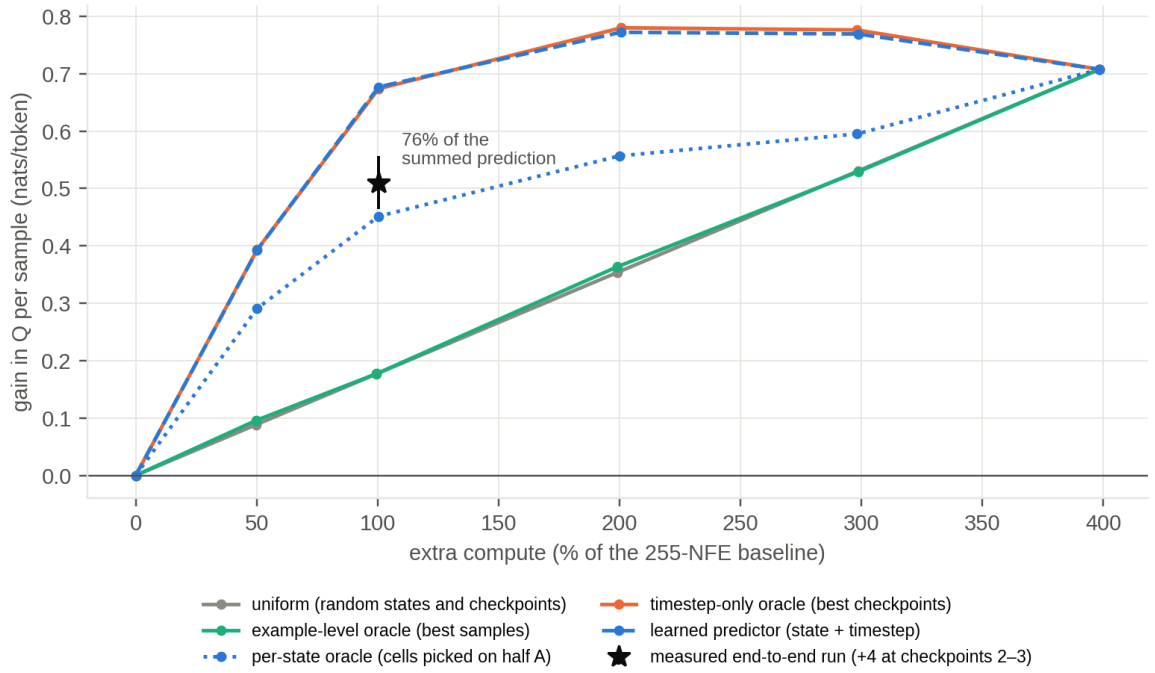


Figure 3. Quality gain per sample at a fixed extra-compute budget (+4 unit). Policies are chosen on half the continuations and scored on the other half. The learned state-aware allocator (dashed) lies on top of the timestep-only schedule (orange); per-sample and per-state selection add nothing. The star is a full end-to-end run, which achieves 76% of the curve's additive estimate.

5. The entropy confound and what remains unresolved

GPT-2 NLL falls when samples become less diverse, so an NLL gain is only a quality gain if diversity is held fixed. Here, every large gain comes with lower unigram entropy of the samples:

Sampler (all 1,024 samples, paired)	NFE	ΔQ (nats/token)	GenPPL	Δ entropy (nats)
Production baseline	255	–	60.2	– (4.313)
+1 at every checkpoint	509	+0.276 [0.236, 0.314]	45.8	−0.069 [−0.086, −0.052]
+4 at checkpoints 2–3 only	511	+0.510 [0.464, 0.556]	36.4	−0.172 [−0.193, −0.150]

Per checkpoint, the +4 branch lowers entropy by 0.146 at checkpoint 2 and 0.027 at checkpoint 3, and across states the size of the gain correlates with the size of the entropy drop (r from −0.17 to −0.66, negative at every checkpoint). By the rule used in our earlier compute-allocation study, an entropy change beyond 0.02 means an NLL change cannot be read as quality on its own; every high-gain policy here exceeds it. A GenPPL of 36.4 at entropy 4.14 is therefore *not* directly comparable with 60.2 at 4.31, and we do not claim that mid-trajectory compute makes samples better.

What survives. The timestep pattern itself is robust, and the small between-state spread at checkpoint 3 persists after a linear entropy adjustment (τ 0.120 → 0.111). **What is unresolved.** Whether the mid-trajectory gain is a genuine quality improvement or mostly a move along the quality–diversity trade-off. The study used one model, one sampler setting, one kind of extra compute and a single quality metric (GPT-2 NLL).

6. Conclusion and recommended next experiment

The value of extra inference compute in CoBit-S is mainly a function of *when* it is spent along the denoising trajectory, not of *which* sample or state receives it. This supports, at most, a fixed timestep-dependent compute schedule. It does not support building state-dependent routing, mixture-of-experts or token-level adaptive compute: the opportunity is small, poorly predictable and adds nothing over the schedule.

Recommended next experiment. Test the timestep schedule at matched entropy before adopting it: compare extra self-conditioning passes in steps 64–127 (and none after step 160) against the baseline sampler with an entropy lever adjusted so that both produce the same entropy. In the earlier study the churn parameter γ barely moved entropy, so another lever such as the churn noise scale or a decoding temperature is needed. Report an entropy-insensitive metric such as MAUVE alongside GPT-2 NLL. If the gain survives, adopt a fixed per-timestep schedule; if it disappears, the mid-trajectory effect was an entropy trade-off.

> **Takeaway**
>
> Extra compute in CoBit-S matters mostly by *when* it is spent: concentrating it in the middle of denoising lowers GPT-2 NLL far more than spreading it evenly. Those gains coincide with lower sample entropy, so they are not yet established quality gains. State-dependent or token-level adaptive compute is not justified by these data: the per-state signal is small, hard to predict, and adds nothing over a simple timestep schedule.